

OmniPurge-40K：架构演进与核心决策备忘录 (PPT 素材库)

文档目的：记录在黑客松产品架构设计阶段，面对“红队压力测试”时我们遇到的工程痛点，以及最终落地的决策与背后的原因。这些内容是答辩路演时展示“技术深度”、“工程务实精神”和“极客巧思”的核心弹药。

1. 迭代流转机制：从“云端 ReAct 博弈”到“本地启发式预抓取”

- **初始设想：**依靠大模型云端的 Agent 循环能力，让 AI 不断阅读报错、下发读取文件指令、分析、再下发，试图还原完整事故上下文。
- **遇到的痛点：**在黑客松 48 小时内，Prompt 极易产生幻觉，模型输出稍微偏离 JSON 格式，本地 CLI 就会崩溃；且多次网络请求在百人赛场极易超时熔断。
- **最终决策：**本地启发式预抓取 (**Local Heuristic Pre-fetch**)。
- **PPT 卖点 (Why)：**我们摒弃了华而不实的云端黑盒循环，将寻找深度上下文的工作下沉到了本地确定性的正则表达式中。只需提取报错行的异常变量，向前追溯 20 行或正则锚定赋值点，一次性拼装出高浓度上下文。用 1 次 API 请求的稳定性，达到了过去 5 次 ReAct 循环才能实现的深层排障效果。

2. 代码审查验证：从“大模型互审”到“真实的本地静态降维打击”

- **初始设想：**设计 Agent A (写代码) 和 Agent B (审查代码)，通过 ClawPro 让它们互相博弈。
- **遇到的痛点：**大模型做静态代码审查（如判断资源是否关闭、是否存在空指针）是主观且不稳定的，容易导致完全正确的代码被误判打回，陷入死循环。
- **最终决策：**javac 降维打击 (**Deterministic Local Post-processing**)。我们没有打包沉重的 Checkstyle JAR 包，而是利用宿主机自带的 javac 编译器。AI 修完代码，本地直接拉起 javac 尝试编译。
- **PPT 卖点 (Why)：**真正的工程工具从不相信概率。我们把虚无缥缈的“AI 审查”，升级为 100% 工业级、字节码级别的 JVM 编译器校验。AI 只有说服了本地真实的编译器，纯洁印章才会落下。零外部依赖，极度轻量，却极度硬核。

3. Java 复杂堆栈解析：从“笨重白名单”到“启发式倒推定位”

- **初始设想：**让用户在 CLI 里配置一个“包名白名单”，防止提取到 Spring 或 JDK 底层框架的报错行号。
- **遇到的痛点：**违背了黑客松“一键执行”、“即插即用”的极速原则。
- **最终决策：**自底向上倒推过滤 (**Bottom-up Backtracking + Depth-limited File Check**)。
- **PPT 卖点 (Why)：**Java 堆栈极长且常伴随 Caused by: 嵌套异常。我们直接从堆栈底部逆向扫描，结合限制深度（最多3层）的本地 OS 文件探测。既能精准穿透表层乱象抓取真正的根因文件，又彻底避免了递归扫描导致的卡顿。实现了“零配置、零延迟”的工业级现场定位。

4. 演示现场防翻车设计：YOLO 战术与防呆机制

- **痛点 A：现场断网熔断 API**
 - **决策：**YOLO 战术（生死看淡）。摒弃复杂的三层高可用降级逻辑。Hackathon 需要的是一发入魂的 Demo，如果断网则直接抛异常，将省下的 3 小时开发时间 All-in 到终端视觉特效（纯洁印章）上。
- **痛点 B：多语言正则带来的“贪婪匹配乱穿”**

- **决策：**YOLO 单语言假设。MVP 阶段战略性放弃多语言，将核心锚定为受众最广、类型最严格的 Java，确保 Demo 极度纯粹稳定。
- **痛点 C：演示时忘记打管道符 (stdin block)**
 - **决策：****Poka-yoke** 防呆设计。引入 Python 的 `select` 模块，为标准输入加装 3 秒超时监控。如果由于紧张少敲了 `cat error.log |`，程序不会死锁卡住，而是优雅提示，挽救可能冷场的 Demo 环节。
- **痛点 D：大模型持续发疯与终端渲染撕裂**
 - **决策：****极限兜底与收敛 (Graceful Degradation)**。若 `javac` 连续 3 次打回失败，系统不崩溃，直接输出最后一版代码供手动修改；废弃易引发 macOS 光标残影的 `rich.live`，降级为最古老的 `print + sleep`。将容错率推向极致。

5. 项目核心价值总结 (电梯演讲 / One-Liner)

OmniPurge-40K 创新性地将“本地启发式上下文预抓取”与“零依赖的确定性编译器校验 (`javac`)”相结合，彻底消除了大模型修复深层 Bug 时的视野盲区与幻觉黑盒。我们将不可控的 AI 代码生成，转化为 **100% 工业级可靠且极具视听沉浸感的终端自动化排障神龛**。

OmniPurge-40K：全链路灵感与数据流转模型

[阶段 1：亵渎捕获] → [阶段 2：逆向寻根] → [阶段 3：圣洁重构] → [阶段 4：降维盖章]

阶段 1：亵渎捕获 (Input Trigger)

- **工程现实：** 研发测试环境或本地终端抛出未经处理的 Java 异常运行时堆栈 (StackTrace)，或者生产环境日志出现了高危 Error。
- **双模触发架构：** 兼顾本地单兵作战与企业协同。
- **亚文化包装：** 「凡俗罪孽显现」—— 肮脏的、未经二进制洗礼的代码在遭到数据践踏后，机魂发出痛苦的哀嚎，亚空间能量 (Bug) 外溢。
- **输入形式：**
 - **CLI 模式 (本地神龛)：** 开发者通过管道流式输入 `cat error.log | purge` 或手动在终端交互界面粘贴报错 (内置 `select` 模块的 3 秒超时防呆检测，若未检测到管道流则优雅打印红色警告并退出，避免演示终端死锁)。
 - **Webhook 模式 (星语者阵列)：** 作为常驻服务或云函数暴露 API，接收业务系统 (或日志平台) 自动推送的异常 Payload。

阶段 2：逆向寻根 (Context Analysis)

- **工程现实：**
 - **CLI 模式：** 1. 文本流解析器采用 **YOLO 单语言假设**，专注于 Java 堆栈，**自底向上逆向扫描** (精准锁定 `Caused by` 根因)。2. 结合限制深度的本地文件判定，穿透底层框架表象，提取业务代码的类名与行号。3. **本地启发式预抓取 (Heuristic Pre-fetch)：** 读取锁定文件截取报错前后上下文，并向下溯源关键变量赋值点。4. 将其封装为增强版 Context Window，以单次 API 请求消灭大模型的视野盲区。
 - **Webhook 模式：** 采用方案 1，推送方在 Payload 中主动附带上下文源码。为应对企业涉密合规，需在推送节点引入“认知过滤 (Scrubbing)”机制，自动剔除注释与字符串硬编码等敏感信

息。

- **商业演进 (Roadmap / PPT 大饼):** 记录方案 2——未来计划授权打通企业级 Git/GitLab API, 实现仅凭堆栈坐标的云端源码自动寻址。
- **亚文化包装:** 「寻回机械心智」——技术神甫越过表面的痛苦症状, 将神圣的光标直接探入代码的底层骨架, 剥离出遭受凡俗污染的局部二进制坏死组织。

阶段 3: 圣洁重构 (LLM Logic Execution)

- **工程现实:** 采用 **Agent A (云端机魂) + 本地确定性后处理器 (静态审判庭)** 架构, 确保演示的 100% 稳定性:
 - 1. **Payload 组装:** 本地工具将【增强版 Context Window】封装后, POST 请求至 ClawPro 云端 API 或 CodeBuddy (采用 **YOLO 网络策略**, 坚决不写复杂的降级与缓存代码, 生死看淡, 将开发精力全部 All-in 终端表现力)。
 - 2. **Agent A (机魂重铸节点):** 扮演技术神甫, 一次性生成带机械教风格润色的修复代码, 内置“三大代码验收基准”的 System Prompt。
 - 3. **确定性后处理 (本地审判庭 / javac 降维打击):** 废弃不稳定的 LLM 互审与笨重的 Checkstyle JAR。CLI 正则提取类名, 将代码写入系统临时目录, 隐蔽触发宿主机 **javac** 进行真实的 JVM 级别编译校验。
 - 4. **闭环拦截与极限兜底:** 若 **javac** 编译失败, 提取 **stderr** 的精确报错信息注入 Prompt 打回重写 (上限 3 次)。若 3 次均失败则触发降级, 程序不崩溃并展示最终版代码; 若全绿通过, 流转至终端渲染。这种零依赖的“真实工程体检”极具说服力。
- **亚文化包装:** 「万机神的二进制启示」——将坏死的源码送入圣洁的 AI 熔炉, 在大模型算力的锤炼下, 以帝国最严苛的法典为标准, 重铸残缺的逻辑。

阶段 4: 降维盖章 (Terminal Final Rendering)

- **工程现实:**
 - **CLI 模式 (MVP 限定演示环境):** 为保证 Hackathon 路演的视听冲击力, 当前版本深度适配并限定于 macOS/Unix 终端展示。1. 控制台输出闪烁红色控诉文本 (保留原始报错日志, 不触发清屏)。2. 绿色逐行异步输出重构后契合对应语言特性的完美代码。3. 巨大的血红色 **ASCII Art 纯洁印章**降级为最简的逐行浮现 (采用最稳妥的 **Console.print + time.sleep**, 摒弃容易产生光标残留的 **rich.live**)。4. 落地瞬间打印红色 **[ALERT]** 产生视觉震颤, 并隐蔽输出 **\u0007** 蜂鸣器彩蛋 (触发即受机魂眷顾, 静音亦不阻碍进程)。
 - **Webhook 模式:** 将重构后的代码、修复建议以及战锤风格的诊断报告格式化为 Markdown 卡片, 通过 WorkBuddy 的 API 推送至指定的企业微信群/开发者账号, 完成自动化告警与修复闭环。
 - **商业演进 (Roadmap / PPT 大饼):** 引入跨平台终端渲染引擎 (如 Python **rich** 结合独立音效库), 彻底解决 Windows 与 Unix 终端在 ANSI 转义码与硬件调用上的差异, 实现全平台统一的沉浸式神龛体验。
- **亚文化包装:** 「机魂大悦·纯洁印章固化」——审判终结, 技术神甫在完全纯净的代码块上烙下万机神不可磨灭的血红圣印, 宣告该模块已获得永久免于宕机的圣洁祝福。

OmniPurge-40K 极限铸造路线图 (Execution Roadmap)

为了在黑客松时限内稳步推进, 本路线图规定了具体的插件依赖、开发模块划分及调试路径。

步骤 0: 铸造厂环境准备 (Environment & Setup)

1. **确认基础环境**：确保本地开发与演示终端安装了 **Python 3.10+**，且配置了 **Java JDK (javac - version)** 可用，用作本地审判庭。
2. **安装核心依赖**：无需沉重的本地扫描引擎，仅需两个轻量级 Python 库。

```
pip install requests rich
```

3. **创建目录结构**：
 - **src/main.py** (CLI 主入口与流程编排)
 - **tests/TestApp.java** (作为制造错误的靶机代码)

步骤 1：第一突击阶段 —— 亵渎捕获与逆向寻根 (约 1-2 小时)

目标：跑通 **cat error.log | python main.py**，能精准截取出真实的 Java 业务代码上下文。

1. **防呆输入 (Poka-yoke)**：在 **main.py** 写入 **select** 模块，监听 **sys.stdin** 3秒，无输入直接打印红色警告退出。
2. **正则嗅探**：编写基础正则提取报错日志中的 Java 文件名和行号。
3. **启发式倒推 (Heuristic Backtracking)**：自底向上（倒序）遍历报错行以锁定 **Caused by** 根因。用 **os.path.exists()** 沿着当前目录向下搜寻（限制深度最大 3 级防止卡顿），锁定真实的业务代码文件。
4. **上下文预抓取**：读取该 **.java** 文件，以行号为基准截取前后 20 行，同时正则检索该行变量的初始赋值点，拼接上下文。

步骤 2：圣洁重构 —— 机魂 API 与 javac 本地审判庭 (约 2-3 小时)

目标：发给云端大模型修复，修复后在本地触发真实 **javac** 编译校验。

1. **打通 Cloud API**：编写 HTTP POST 请求对接大模型（引入 **--mock** 参数返回硬编码字典供前期调试，减少多文件维护）。
2. **javac 降维打击**：编写后处理器函数，用正则 **(public\s+)?class\s+(\w+)** 提取 AI 代码中的类名，为防 CWD 漂移，将代码写入系统临时目录 **tempfile.gettempdir()/ {ClassName}.java**。
3. **触发验证**：调用 **subprocess.run(["javac", "{TempDir}/{ClassName}.java"])**。
4. **闭环重试与降级**：如果 Exit Code 不为 0，提取 **stderr** 报错，明确拼装进下一轮的 **Prompt** 中 (**[JAVAC_ERROR]: ...**)，发起重试请求（最多 3 次）。若 3 次均失败，打印黄色警告并强行输出最后一次代码。

步骤 3：降维盖章 —— 终端特效渲染 (约 1-2 小时)

目标：震撼评委的视听神经。

1. 利用 **rich.console.Console** 控制终端输出。
2. 保留原有堆栈，下方红字闪烁：“**[警告] 检测到亚空间逻辑污染，开始净化...**”。
3. 使用简单的 **time.sleep(0.02)** 配合高亮，稳定逐行绿色浮现重构代码（摒弃 **live** 防止终端时序冲突）。
4. 逐行打印血红色纯洁印章 ASCII Art。
5. 落地瞬间 **print("[ALERT]\a")** 触发视觉震颤与硬件蜂鸣。

步骤 4：联调与彩排 (最后冲刺)

1. 替换真实的大模型 API。
2. 用 `tests/TestApp.java` 制造空指针或未闭合流，实战录制一镜到底的排障修复视频。